

PaxosLease Revisited: A Checked Model of Diskless Distributed Leases

Márton Trencsényi (mtrencseni@gmail.com)

Abstract

PaxosLease is a protocol by which a quorum of acceptors grants time-bounded exclusive ownership with no durable acceptor lease state and no disk write on the lease acquisition path. This paper gives a precise, machine-checked statement of the protocol and of its standard use, electing a Multi-Paxos leader. Model checking the original protocol with TLC changes two rules of its acceptor: one is required for safety, the other allows shorter restart quarantines. The protocol is formalized in TLA+ and checked by TLC, its timing arithmetic is proved in TLAPS, and an executable Python model demonstrates the distributed algorithm for human readers.

1. Introduction

PaxosLease [1] is a protocol by which a quorum of acceptors grants time-bounded exclusive ownership, diskless in the sense that acceptors do not write to disk, and proposers write to disk only once, at startup. This paper gives a precise, machine-checked statement of the protocol and of its standard use, electing a Multi-Paxos leader that recovers once per epoch and then commits with single-round appends.

A lease is a lock whose ownership is bounded by time and expires automatically [3], so a holder that has crashed or become unreachable before giving up the lease does not block progress in the distributed system. A process may act as owner only until its lease expires, and a process that granted the lease refuses conflicting grants until its own exclusion interval has expired. Exclusion is the property that at any time¹ at most one process has the authority to act as owner. PaxosLease achieves exclusion with ballots and process identities carried in messages, local timers, and majority quorums. Clocks need not be synchronized, but the intervals measured by different processes must have bounded error.

PaxosLease was described in 2012 [1] and implemented in the open source systems Keyspace [14] and ScalienDB [15]. Neither the description nor the implementations were model checked, and both contain subtle bugs, in the algorithm and in the C++ code. This paper states the protocol precisely, checks it in TLA+, and changes two rules of its acceptor: an acceptor must never replace another owner's live lease, even under a higher ballot (Section 6), and a restarted acceptor need stay silent only for the proposer attempt duration (Section 4). Section 7 and Appendix D

discuss the bugs in the 2012 description and in the implementations, each supported by a TLA+ model or an executable witness.

Section 2 describes the protocol in plain terms, Section 3 states the system model and the algorithm, Section 4 gives the safety argument and the quarantine bound, and Section 5 summarizes the checked models. Section 6 gives the safety violations of the original formulation, Section 7 audits the historic implementations, and Sections 8 and 9 discuss related work and limitations. The appendices give the leased-leader algorithm, the checked invariants, the executions with overwriting acceptors, the audit in full, the evidence inventory, and how the paper was written.

2. How PaxosLease Works

PaxosLease grants one process a time-bounded, exclusive right to act as owner of a (potentially abstract) resource. The idea is to borrow the structure of Paxos, its ballots, its two rounds and its majorities of acceptors, but for a value that is not a log entry: a statement of the form “node 3 owns the lease, for the next seven seconds.” A lease has a bounded validity interval, so an acceptor need remember it only until it expires, and can keep it in memory; PaxosLease acceptors do not write to disk.

An acquisition is two rounds. A proposer picks a ballot number that nobody has used before and sends it to every acceptor in a Prepare message: the acceptors promise to reject any smaller ballot from now on, and respond whether they currently hold a live lease for another owner. If a majority of acceptors answer, and none of them reports a live lease belonging to another owner, the proposer sends a second, Accept message² asking the acceptors to record

¹This paper implicitly assumes Newtonian relativity, where “at any time” has an absolute, frame-independent meaning. A companion paper explores the modification of PaxosLease to special relativity with moving inertial observers.

²The 2012 paper [1], and the Keyspace [14] and ScalienDB [15] implementations built on it, call this second round *propose*. This paper uses *accept*, the name the round carries in [2], and keeps *propose* only where the text describes the original pseudocode.

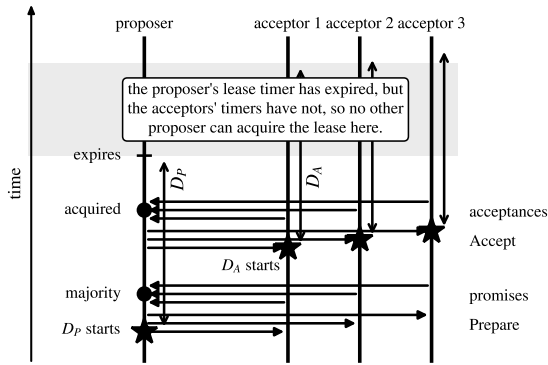


Figure 1: One acquisition in PaxosLease, as a message-sequence diagram. The proposer's timer D_P starts when `Prepare` is sent, each acceptor's timer D_A starts when it accepts, and D_P ends first.

it as the owner for a fixed duration. An acceptor that holds a live lease for another owner refuses to accept until its own timer has expired. When a majority confirm, the proposer is the owner, and it informs the learners. Figure 1 shows the two rounds as a message-sequence diagram.

Any two majorities of the same set share at least one member, so two proposers cannot both collect a majority of acceptances for leases that overlap in time: the acceptor the two majorities share holds the first lease for at least as long as its owner may act on it, and refuses the second. The first round lets a proposer find a live lease before it asks for acceptances it would not get, and the ballots let acceptors discard the messages of superseded attempts.

What Paxos does not have to reason about, and this protocol does, is time. Two obligations follow. First, an owner's authority must end no later than the acceptors' exclusion; if it ended later, a second proposer could acquire while the first still believes itself the owner. Second, an acceptor that crashes loses its promises and its accepted lease, and if it answered again at once it could help a second proposer acquire a lease that is already held. To address this, a restarted acceptor refuses all lease-layer messages for a fixed quarantine interval on restart.

A lease is renewed by running the same two rounds again, under a higher ballot, before the current one expires. A lease may also be released early; this is an optimization that speeds up the next acquisition, but is never required for safety.

The standard use of PaxosLease is to elect a Multi-Paxos leader. While a proposer holds the lease no other proposer can acquire it, so no other proposer runs the Paxos prepare round (Phase 1). The holder runs Phase 1 once for the whole log and then commits each value with the accept round (Phase 2) alone: one round trip and one durable write per acceptor instead of two, on a fast path that competing ballots do not preempt. The lease also lets the leader serve reads from its local state without a Paxos round, since no other process can commit writes while it holds the lease.

[Appendix A](#) states the leader lifecycle and the composition model that checks it.

3. PaxosLease Algorithm

A reader who wants to implement PaxosLease should work from this definition, and not the original in [1]. We state the algorithm as numbered steps. Every step names, in brackets, the action of `tla/spec/PaxosLease.tla` that formalizes it. The TLA+ module, not this prose, is the normative statement of the PaxosLease algorithm.

3.1. System Model

The system contains a finite set of proposers and a finite set of acceptors. A quorum is a set of acceptors such that any two quorums share at least one acceptor; specifically the paper uses majorities.

Processes are not Byzantine, but any process may crash. A crashed acceptor loses its promised ballot and its accepted lease. A proposer keeps one word on stable storage, a restart counter incremented at every start, which makes its ballots unique across restarts.

Messages may be delayed, reordered, duplicated, or lost, and a message sent to a process before it crashed may be delivered after it restarts. The models also check the connection-oriented alternative, in which a crash discards the messages addressed to the crashed process ([Appendix D](#)).

Clocks are not synchronized, and messages carry no timestamps. Each process measures elapsed time on its own clock, and the rate of every clock is within a factor $1 \pm \rho$ of real time, with $0 \leq \rho < 1$.

3.2. State

Proposer state is per attempt and volatile, with one durable exception.

- the current ballot b , and the phase of the attempt (idle, preparing, accepting);
- the pending deadline t_d of the attempt, set when `Prepare` is broadcast (T1) and checked by P3 and P4;
- the renewal base: the ballot of the lease this attempt renews, or none if the attempt is a fresh acquisition (P1, P2);
- the identities of the acceptors that have responded in the current attempt, kept as a set rather than a count (P2);
- when active, the ballot of the active instance, the authority deadline, and the quorum certificate that established the lease;
- the restart counter, which is the one piece of proposer state written to stable storage ([Section 3.1](#)).

Acceptor state is entirely volatile, which is what makes the protocol diskless.

- the highest ballot promised since the last restart;
- the accepted lease instance, if one exists, and the local deadline until which it excludes conflicting grants; the accepted lease is *live* until that deadline passes on the acceptor's clock.

A lease instance is identified by (*owner, ballot*), where the ballot is globally unique, also across proposer restarts. The owner is a proposer incarnation: the proposer together with the restart counter its ballots carry (P1), so a proposer that restarts is a new owner. This is required for safety after restarts.

3.3. Durations

PaxosLease uses three fixed timeout durations, D_P , D_A and D_Q . Each is measured on the clock of the process that uses it, and ρ bounds the rate error of every clock (Section 3.1).

- D_P , the proposer lease duration: an attempt whose Prepare is sent at time t can make its proposer active only until $t + D_P$ (P1, T1).
- D_A , the acceptor lease duration: an acceptor that accepts a lease refuses conflicting grants for D_A time after the acceptance (A2).
- D_Q , the restart quarantine: a restarted acceptor refuses every lease-layer message for D_Q time (A4).

The timing rules T2 and T3 require D_A and D_Q to be at least $D_P \cdot (1 + \rho)/(1 - \rho)$.

3.4. Proposer

P1 [StartAcquire]: Choose a fresh ballot b , never used by any proposer, including this proposer's own earlier incarnations. A ballot is a triple (*counter, restart, node*) with counter-major comparison: ballots compare lexicographically, with *counter* most significant and *node* as the final tiebreak. The *counter* increments per attempt, and *restart* is durably stored and incremented at every process start, so no ballot is reused across proposer restarts. Safety needs only this uniqueness, together with the *restart* component, which identifies the incarnation for A2. The counter does not order a restarted proposer's ballots above its earlier ones, since $(1, restart+1, node) < (100, restart, node)$; for liveness, a proposer raises its counter above any ballot it observes in responses, which requires rejections that carry the promised ballot (A1). Record as the attempt's *renewal base* the ballot of the lease this proposer currently holds, or none if this is a fresh acquisition; P2 tests self-owned reports against it. Assign the pending deadline $t_d := now + D_P$ as stored state, and broadcast Prepare(b).

P2 [DeliverPromise]: On a promise for the current b , record the responder's *identity* and call the response *open*

if it reports no live lease, or, for a renewal attempt, if it reports exactly the lease being renewed (the renewal base captured at P1) while the proposer still holds that lease. Quorums are sets of distinct acceptors (not just counts of messages, see Section 7, fourth finding). A self-owned accepted lease under any other ballot may have been installed by an Accept request an abandoned attempt left in flight, and it blocks like a foreign lease. With A2's acceptor-side rule the qualifier is not needed for exclusion (Section 4); it is needed with acceptors that overwrite (Appendix C).

P3 [SendAccept]: When promises from a quorum are recorded and $now < t_d$: broadcast Accept(b , self).

P4 [DeliverAccepted]: When Accepted responses for b have arrived from a quorum of distinct acceptors and $now < t_d$, become active until t_d , record the quorum as the certificate, and broadcast LearnChosen(self, t_d); later responses under b are discarded.

P5 [AbandonAttempt, then P1]: An attempt that cannot proceed is abandoned, at the latest when $now \geq t_d$: the proposer returns to idle, alive and keeping whatever lease it already holds, and retries with a fresh, higher ballot from P1. Messages of the abandoned attempt may still be in flight, and the ballot comparison in P2 and P4 discards every response belonging to it. Retry does not advance the durable restart counter; only a crash and restart does.

P6 [Release, optional]: To release, first stop acting as owner, then broadcast Release(b_a), where b_a is the ballot of the currently acquired lease. An attempt in progress may be abandoned or left to complete; either is safe (Section 4). Release is an availability optimization, not a safety requirement.

Messages may be lost, so in practice a proposer abandons an attempt and retries (P5) after a timeout, or as soon as the responses it has collected show that it cannot obtain a majority.

A renewal is a new acquisition by the current owner, under a new, higher ballot. At P1 the attempt records the ballot of the lease it extends as its renewal base.

3.5. Acceptor

A1 [DeliverPrepare]: On Prepare(b): if $b \geq$ the promised ballot, set promised $:= b$ and reply with the currently live accepted lease, reporting an expired lease as no lease; otherwise reply rejecting, or not at all. A rejection is a speed optimization: it lets a proposer learn early that it cannot obtain a majority. So that proposers can raise their counters (P1), it should carry the promised ballot. Safety does not depend on rejections: an acceptor that stays silent refuses just as effectively, and the proposer's own timeout ends the attempt either way.

A2 [DeliverAcceptReq]: On Accept(b , o): if $b \geq$ promised and the acceptor holds no live lease of an owner other than o , set promised $:= b$, record the lease (o , b) with exclusion deadline $now + D_A$, and reply accepted; otherwise reject. The second condition is the acceptor-side rule

of [Section 6](#): without it, a renewal across a single acceptor restart gives two owners, and so does early release. Owners are compared as proposer incarnations. A restarted proposer's ballots may be lower than its previous incarnation's, and a delayed `Accept` request of the previous incarnation must not replace the new incarnation's lease ([Section 6](#)).

A3 [DeliverRelease]: On `Release` from o with ballot b : clear the accepted lease only if it matches the message's (o, b) exactly. Matching on the owner o alone leads to a safety violation ([Table 1](#), 36-state trace). Exact matching is necessary but not sufficient: without A2's refusal rule, a stale `Accept` can reinstall a released instance over another owner's live lease, and the stale `Release` then clears it, exclusion deadline included ([Section 6](#)).

A4 [RestartAcceptor]: On restart after losing volatile state: refuse every lease-layer message, prepare, accept, and release alike, until $now + D_Q$.

3.6. Learner

L1: Activation is made application-visible through learners. The owner's own learner installs the absolute deadline t_d , and only the owner acts on the lease. A remote learner depends on an assumed bound on message delivery time, so its view of the lease is only a hint ([Section 7](#), sixth finding): it may report a lease that has already expired or passed to another proposer. This is safe as long as the view is used only as an optimization, for example to keep the local proposer from starting an acquisition while a lease is reported. In the worst case a proposer waits longer than necessary, or starts an attempt that the acceptors refuse, and the system is without a leader for longer time between leases.

3.7. Timing Rules

This section enumerates the PaxosLease timing constraints as named rules for proposer attempts, lease containment, and acceptor restart quarantine.

T1 (proposer lease timeout): The pending deadline t_d is assigned when `Prepare` is broadcast (P1) and checked in the transitions that use the attempt's evidence (P3, P4). Safety requires only that the timer start no later than the first acceptor accepts the lease ([Section 4](#)). Starting it when `Prepare` is sent is needed only for the overwriting acceptors of [1], for which a later start is unsafe ([Appendix C](#)).

T2 (acceptor lease timeout): An owner's authority must end no later than the exclusion interval of every acceptor in its certificate. With equal clock rates $D_P \leq D_A$ suffices, since the proposer's timer starts no later than any acceptance (T1). With rate bounds $r_{min} = 1 - \rho$ and $r_{max} = 1 + \rho$ the condition is

$$D_P \cdot r_{max} \leq D_A \cdot r_{min},$$

which covers a proposer clock running slow and an acceptor clock running fast; the TLAPS obligation `DriftContainmentArithmetic` proves the arithmetic.

T3 (acceptor restart quarantine): With equal clock rates, $D_Q \geq D_P$ suffices. With the rate bounds of T2 the condition is

$$D_P \cdot r_{max} \leq D_Q \cdot r_{min}.$$

Neither the acceptor lease duration D_A nor the maximal lease time appears in it; [Section 4](#) shows that it suffices and, by checking in two-acceptor acquisition configurations, that it is tight even when $D_P < D_A$.

4. Safety Argument

Theorem. At any time, at most one proposer is active.

The theorem is the invariant `LeaseExclusivity` of `tla/spec/PaxosLease.tla`; every configuration of `PaxosLeaseChecked.tla`, the module that assumes T3 and A2's rule, passes it. The following argument assumes $\rho = 0$ for simplicity; it is not a mechanized proof.

Argument. Suppose that two different proposers p and q are both active at time t_v , violating safety. Each is active under a lease instance, (p, b_p) and (q, b_q) , whose `Accepted` responses came from a quorum, its certificate. Any two quorums intersect, so some acceptor a is in both certificates and accepted both instances. Name the proposers so that a accepted (p, b_p) first, at time t_p , and (q, b_q) later, at time t_q . Since q became active only after a 's response, $t_q \leq t_v$.

By timer containment, a 's exclusion deadline for (p, b_p) is no earlier than the end of p 's authority under that instance, which lies after t_v , so on a 's own clock the lease was still live at t_q . A2 then refuses (q, b_q) unless a lost its accepted lease of p between t_p and t_q . Only an `Accept` request, a release, and a crash change an acceptor's accepted lease before it expires.

- An `Accept` request replaces a live accepted lease only when it names the same owner, the same incarnation of p (A2), and only under a ballot no lower than a 's promised ballot, which is at least b_p from t_p on. The replacement is therefore an accepted lease of that incarnation under a ballot of at least b_p , with a later exclusion deadline, and never one of q , so a still holds a live accepted lease of p at t_q and refuses (q, b_q) .
- A release clears an accepted lease only when it names that exact instance (A3), and a proposer releases only the instance it is active under, after it has stopped acting under it (P6). A release of (p, b_p) before t_v contradicts p being active under (p, b_p) at t_v , since an instance activates at most once (P1, P4). A release of a lower instance of the same incarnation matches nothing, since from t_p on a 's accepted lease carries a ballot of at least b_p (first case). A release of a higher instance contradicts it as well: p was active under that higher instance when it released it, before t_v , and an incarnation's active ballot never decreases, so it cannot be active under (p, b_p) at t_v .

- If a crashed between t_p and t_q , it could not accept (q, b_q) until $D_Q \geq D_P$ after its restart (A4, T3). The pending deadline of (p, b_p) was assigned no later than a accepted it (T1), so p 's authority under that instance ended at most D_P after t_p , no later than t_q , which contradicts p being active at t_v .

Every case contradicts the assumption, so at most one proposer is active at any time. The same argument holds with clock-rate error ρ and the rate-adjusted forms of the inequalities T2 and T3.

The crash case shows that $D_Q \geq D_P$ suffices: the acceptor exclusion duration D_A does not appear, and with A2's rule a forgotten promise imposes no bound of its own. The bound is tight in the checked model: at two separated duration pairs, a quarantine of D_P passes exhaustively and one of $D_P - 1$ yields a two-owner trace (Table 2), in two-proposer, two-acceptor acquisition configurations.

The argument uses neither Prepare responses nor P2's renewal qualifier, so with A2's rule these only make acquisition efficient; with overwriting acceptors they are needed for safety (Appendix C).

This is a safety argument only. Liveness requires eventual message delivery, sufficiently accurate timers, and a period during which a quorum of acceptors stays available; as in Paxos, two proposers can starve one another with ever higher ballots, which implementations prevent with randomized backoff [1].

A2 compares owners as incarnations. A proposer's attempt counter is volatile, so a proposer that restarts starts it again under a higher restart component, and its new ballots may be lower than its old ones (P1). If A2 compared owners by proposer alone, a stale message of the previous incarnation could pass as the current owner's. Proposer p_1 acquires under ballot 2 through a_1 and a_2 and releases, with its Accept request and its Release to a_3 still in flight. It then crashes and restarts, a_2 crashes and restarts into the full quarantine, and p_1 acquires again under the lower ballot 1, through a_2 and a_3 . The stale Accept request of ballot 2 reaches a_3 ; its ballot is higher and its proposer is the same, so a comparison by proposer alone replaces p_1 's live lease with the released instance, and the stale Release then clears it. Proposer p_2 acquires through a_1 and a_3 while p_1 is active. TLC finds the execution in a 43-state trace on the variant `NodeOwner.tla` (Table 1), and the same directed search on the specification, which compares incarnations, exhausts its space with no violation. The reference model reproduces an execution of the same shape (release-across-proposer-restart). The clause is needed because of the counter-major layout of P1, kept from [1]. With the restart counter above the attempt counter, a proposer's ballots would increase across its restarts and A2's ballot guard alone would reject the previous incarnation's stale Accept request, but a proposer that had restarted fewer times could then outbid another only by raising its own restart counter, a durable write.

5. Checked Models and Tests

Unlike the 2012 formulation, the protocol of Section 3 comes with a formal TLA+ specification that TLC checks exhaustively in finite configurations, with the timing arithmetic proved in TLAPS, and with an executable reference model that encodes the same rules a second time. The accompanying GitHub repository (Appendix E) contains:

- the TLA+ specification and its weakened variants;
- the TLAPS proof module;
- a Python reference model of the lease protocol and of its composition with Multi-Paxos;
- eighteen structured witnesses that keep every known failure executable, and the tests;
- a single-file runnable demonstration;
- the recorded output of every run.

TLC checks LeaseExclusivity together with the supporting invariants of Appendix B, and Table 3 maps each rule of Section 3 to the model, variant, witness or audit that checks it. Each variant weakens exactly one rule and is generated from the specification by a script, and TLC finds a two-owner execution in each (Table 1). A second TLA+ model projects the protocol the historic implementations ship (Appendix D).

The passing configurations cover acquisition races, acceptor crash and restart, renewal, release, abandoned attempts and a redelivering transport, separately and in combination. The passing checks are exhaustive only within small configurations: at most two proposers, three acceptors, three ballots, one restart per proposer, and a time horizon of three units, with one duration triple (D_P, D_A, D_Q) per configuration (Table 2). Even within these bounds the recorded runs explore more than five billion distinct states, and TLC generates more than thirty billion states along the way; the two largest runs, Retry and Renew+Retry, each exceed two billion distinct states.

6. Safety Violations in the Original Formulation

The 2012 PaxosLease protocol existed as a paper and open source implementations for over a decade with no machine-checked model. The 2012 paper [1] describes the start of the proposer's timer twice and the two descriptions disagree. Its figure draws the deadline stored when Prepare is broadcast, the rule this paper specifies as T1; its pseudocode states the quorum-receipt rule, which Appendix C shows unsafe with the overwriting acceptors that [1] specifies; and its prose is consistent with both.

In [1], an acceptor accepts every Accept request with a high enough ballot, even when it already holds a live lease for another owner: the old lease is simply overwritten. Keyspace and ScalienDB do the same. A2 changes this, and an acceptor refuses to replace a live lease of another owner. The two executions below show why the change is

Variant	Weakened rule	Violation	Trace
LateTimer	timer starts at prepare-quorum receipt	LeaseExclusivity	27 states
OwnerOnlyRelease	release matches owner, ignores ballot	LeaseExclusivity	36 states
ScalarQuorumCounting	quorum counted by message, redelivering transport	LeaseExclusivity	18 states
StaleOwnerOpen	own lease open without the renewal qualifier, retry enabled	LeaseExclusivity	36 states
NodeOwner	A2 compares owners by proposer, not incarnation	LeaseExclusivity	43 states

Table 1: Counterexample variants: full copies of the specification with exactly one rule weakened, regenerated from it and compared against the checked-in files by `make variant-check`. The target `make counterexamples` requires TLC to find the violations of `LateTimer`, `OwnerOnlyRelease`, `ScalarQuorumCounting` and `NodeOwner` on every run, the last through a directed search whose state constraint only prunes behaviors; the `StaleOwnerOpen` search is multi-hour and is recorded evidence, checked against the cited trace length by `make paper-claims`. The `LateTimer` and `StaleOwnerOpen` configurations use the full quarantine bound and overwriting acceptors, the rule of [1]; with A2’s rule both pass.

needed: with overwriting acceptors, each gives two owners at once, even with the timer rule T1 and the renewal check of P2 in place. With A2’s rule, T1’s timer placement and P2’s renewal check are no longer needed for safety; [Appendix C](#) shows what goes wrong without them when acceptors overwrite.

Renewal across an acceptor restart. With three acceptors, one acceptor restart lets a renewal produce two owners. The restarted acceptor has forgotten its promise to one proposer, whose lease rests on an accepted lease it overwrote at another acceptor; after its quarantine it accepts the other proposer’s delayed `Accept` request and completes that proposer’s majority. TLC finds the execution in a 39-state trace, and the same search with A2’s rule finds none ([Appendix C](#)).

Early release across an acceptor restart. A delayed `Accept` request puts a released lease back at an acceptor, over another owner’s live lease, and the delayed `Release` then clears it, so a third acquisition succeeds while that owner is still active. No quarantine length prevents this; TLC finds it in a 40-state trace, and with A2’s rule the search finds none ([Appendix C](#)).

7. Audit of the Historic Implementations

PaxosLease was implemented by this author in two open source systems, Keyspace [14] and later ScalienDB [15]. This audit examines the final commits of both repositories against the checked model and against a checked projection of the shipped protocol, and makes eight findings, which [Appendix D](#) gives in full with the source-level evidence.

1. Both systems implement the quorum-receipt timer rule of the pseudocode, which is unsafe with their overwriting acceptors, and an unrelated retry timeout of $R = 2$ seconds accidentally prevents the late-timer executions.
2. That prevention requires $D_Q \geq \max(D_P, R)$ under timely timeout dispatch, in acquisition configurations without renewal, and the shipped constants satisfy it.

3. The event loop can run an expired attempt’s handlers before the retry timeout fires: TLC finds a two-owner execution under the shipped constants, also when only the quorum-intersection acceptor crashes.
4. Quorums are counted by message rather than by acceptor identity, which is safe only because their TCP transport does not deliver a response twice.
5. Every lease timer reads the wall clock, and neither system defends both of the directions in which a wall clock is unsafe.
6. Remote learners start the lease countdown at message arrival, which is safe only under an undocumented 500 msec delivery bound.
7. A proposer treats any lease it owns as an open response; with the shipped retry timeout and overwriting acceptors, TLC finds a two-owner execution with timely dispatch and no clock error.
8. The activation guard reads the clock twice, and an unsigned subtraction can wrap so that a proposer activates a lease that has already expired.

In summary, both Keyspace and ScalienDB contain bugs through which two proposers can hold the lease at the same time: TLC finds such executions in the checked model of the shipped protocol (third and seventh findings), and wall-clock steps and the unsigned subtraction give further paths to an owner acting past its lease (fifth and eighth findings).

8. Related Work

Leases as time-bounded ownership originate with Gray and Cheriton [3]. Chubby [6] made the lease-plus-Paxos architecture standard practice at scale, and the engineering account of [7] describes how much of the difficulty lies outside the core algorithm. FaTLease [8] solves the same lease-negotiation problem as PaxosLease, but it runs Paxos instances for the lease commands and assumes clock synchrony, while PaxosLease was designed to remove both [1]. The closest relative is Flease [10], from the same

group as FaTLease: decentralized lease coordination without stable storage for the lease state, built on a round-based register.

Raft [4] structures leadership through terms and elections, but its safety, like that of Paxos, does not depend on real-time exclusivity. Production Raft systems that serve reads from the leader without a log round trip add a leader lease, and they inherit exactly the containment and clock-rate obligations of T2. Paxos Quorum Leases [12] let a quorum of replicas serve local reads; they are a read-performance mechanism layered on Paxos rather than a diskless leadership protocol, but rest on the same containment argument.

On the verification side, Chand, Liu, and Stoller [5] give a full TLAPS proof of Multi-Paxos, and Grove [11] is the modern benchmark for mechanized lease reasoning, verifying time-based leases together with crash recovery, reconfiguration, thread-level concurrency, and unreliable networks in an executable key-value store. Lamport’s Paxos papers [2, 9] set the specification style, and the trace-validation work of Cirstea et al. [13] shows how the remaining gap between such models and an implementation can be closed mechanically.

9. Limitations

The main limitations of this work are the following.

1. PaxosLease assumes non-Byzantine processes that follow the protocol and stop acting when their leases expire.
2. Only majority quorum systems³ were checked, although the argument of Section 4 uses only quorum intersection.
3. The acceptor set is fixed. Reconfiguration requires preserving quorum intersection across configurations, or else waiting until leases from the old configuration can no longer matter.
4. The TLC runs are finite-state checks within the bounds of Section 5, with a cap on messages in flight; every configuration with $D_P < D_A$ has two acceptors and neither renewal nor release.
5. **PENDING: renewal and release with two proposers, crash, three acceptors.**
6. The clock model assumes monotonic elapsed-time measurement with bounded rate error. A suspended, paused or live-migrated process can resume after its lease has expired and go on acting as owner.
7. Only the timing arithmetic is mechanically proved. The exclusivity theorem rests on three things together, none of them a proof of it: the argument of Section 4, those arithmetic lemmas, and finite model checking.

³ Alternatives include weighted majorities, grid quorums (a row plus a column), hierarchical majorities of majorities, and flexible quorums with different families for the two rounds.

8. The audit of Section 7 is a reading of the sources against the model, backed by a checked projection and an executable abstraction of the node; we did not run the described executions against the shipped binaries, and the claims about margins assume the shipped compile-time constants.

10. Lessons

Writing this paper taught the author four lessons that may apply to distributed algorithm research and practice beyond the scope of this paper. The first three follow the steps of the development process, from specification to implementation; the fourth concerns automating them.

1. **Formal specification and checks.** Lamport’s standing advice⁴ is to write the algorithm in TLA+ and let the model checker debug it.
2. **Code derived from checked specifications.** Production source code should be written from the checked TLA+ specification, not from pseudocode or prose without tool checks. Both audited systems were written from the pseudocode of [1], which states the timer rule in its unsafe form.
3. **Code re-encoded as a specification and checked.** Software adds networking and messaging code, event handling and control flow, concrete data types for the algorithm’s variables, and other implementation detail, and each of these is an opportunity to introduce a bug into an otherwise correct algorithm. By translating the software implementation back to a specification and then checking it, the risk of shipping unsafe code to production is reduced.
4. **Automation with frontier language models.** The time required to write and maintain a specification, its configurations, its weakened variants and its harness has discouraged engineers from using formal methods. Modern frontier models automate away much of this work. However, extensive human guidance and review are required throughout the process (Appendix F).

11. Conclusion

PaxosLease grants time-bounded exclusive ownership from a quorum of acceptors that keep no durable lease state. Exclusion follows from quorum certificates whose acceptor exclusions outlast the proposer’s authority, from acceptors that never replace another owner’s live lease, where a restarted proposer is another owner, from release that clears only the exact instance named, and from a quarantine of one attempt duration after an acceptor restart.

⁴In December 2009 the author described PaxosLease in an email to Leslie Lamport, who replied that he did not have time to look at the algorithm and suggested coding it in TLA+ and using the model checker to debug it. This paper is that suggestion, carried out seventeen years later. He was right on both counts: the model checker was the proper instrument, and the algorithm did need debugging.

Appendix A. Using PaxosLease in Multi-Paxos

PaxosLease is not Paxos, and the two should not be confused. Paxos solves agreement for log slots, choosing at most one value in each slot [2]. Unlike the lease layer, its acceptors are durable: a promise and an accepted value must each reach stable storage before the acceptor replies, so committing one value in plain Paxos costs two round trips and two durable writes at every acceptor.

The leader optimization removes half of that. A proposer runs Phase 1 once under a single ballot for the whole log rather than for one slot, and the promises it collects reserve that ballot for every slot it will later fill. From then on it appends by running Phase 2 alone, so each new value costs one round trip and one durable write per acceptor instead of two. Phase 1 is amortized, not skipped, and the reservation lasts until another proposer’s own Phase 1 wins a quorum of promises under a higher ballot.

This is what the lease contributes. A proposer holding a PaxosLease renews it, and may go on renewing it indefinitely: until it crashes, until the network prevents a renewal, or until the application hands leadership elsewhere. While it holds the lease no other proposer can acquire the lease, and hence no other proposer will run a Prepare phase, so the single reservation stays valid across arbitrarily many appends and the two round trips of the initial full round are amortized over all of them. Driven to the limit, the amortized cost of committing a value is one round trip and one durable write at each acceptor, which is what makes leased Multi-Paxos a practical way to implement a replicated log.

What the lease does not do is replace Phase 1. A proposer that has just acquired the lease does not know which values earlier leaders may already have had accepted, or chosen, and it must run a full first round to discover them and finish them before proposing anything of its own.

The standard use of the lease, and the one Keyspace and Scaliendb implement, is this classic Multi-Paxos leader optimization. We state the leader lifecycle precisely, in the form of [Section 3](#): each step names its counterpart in the composition model (`tlaspec/PaxosLeasePaxos.tla`). That model abstracts the lease layer to its conclusion: its `AcquireLease` action is enabled only while no proposer is active, so lease exclusivity is a premise there, and its invariants check the lifecycle above the lease; the Python composition runs the lease model underneath.

M1 [AcquireLease]: Win the PaxosLease of [Section 3](#). This starts a fresh *leadership epoch*, the interval over which one proposer holds the lease continuously, across any number of renewals, and it *clears* any recovery standing from an earlier epoch.

M2 [StartRecovery/FinishRecovery]: Run a log-wide Paxos Phase 1 under a fresh ballot. In the Multi-Paxos organization assumed here, and implemented by both audited systems, a promise quantifies over all instances rather than one [2, 9], so the single round both reserves the ballot for the entire log and discovers the accepted values that constrain recovery.

M3 [repair]: For every slot that Phase 1 reports constrained, select the value attached to the highest-ballot `Accepted` response for that slot, re-propose it under the reserved ballot, and learn the results. Phase 1 alone is not cleanup: a value that is discovered but not re-proposed leaves recovery to be finished by whatever proposal next touches the slot.

M4 [BecomeReady]: Only now is the leader ready. Readiness is per epoch: the invariant `ReadyImpliesRecovered` checks that a re-elected leader repeats M2 and M3 rather than relying on a previous epoch’s recovery.

M5 [AdmitOrApply]: Fast mode. Each client command is placed, unaltered, into a new slot with a single accept round under the reserved ballot and no per-instance `Prepare`.

M1 relies on the exclusion theorem that [Section 4](#) establishes for the lease layer; none of M2 through M5 does. Paxos agreement does not depend on lease exclusivity. Acceptors reject lower-ballot accepts regardless of any clock, so even two processes that simultaneously believe themselves leader cannot choose conflicting values, and skipping `Prepare` in M5 is sound because M2 already promised the whole log. The lease adds no safety to this and is needed for none of it. What the lease adds is efficiency and liveness, since with at most one real-time leader the fast path is not repeatedly preempted by dueling ballots, and local admission and lease-guarded reads become possible. One caution on reads: the lease

alone does not make a local read current. A leader serving reads locally needs the lease, this epoch’s completed recovery (M2, M3), and an applied state machine covering the relevant prefix; ownership without recovery reads stale state. Conversely, the fast mode imposes only one obligation on the lease layer: no admission before the leader’s own epoch has completed M3.

The PaxosLease layer and the Paxos layer have a one-way interface, which is what makes them independently reviewable. The PaxosLease layer determines which Paxos proposer may run M2 through M5, and when it may begin. No Paxos transition reads PaxosLease state: a Paxos acceptor deciding whether to promise or to accept consults its own promised ballot and its own accepted values, and nothing else, and it does not know whether the Paxos proposer addressing it holds the lease. No Paxos transition reads a clock: deadlines, durations, drift bounds and the quarantine all live in the PaxosLease layer, and agreement on the log is a function of the message history alone. The optimization therefore changes only the Paxos proposer, which skips the prepare phase while the lease condition holds; the Paxos acceptor and the Paxos learner are unmodified.

Appendix B. Checked Invariants

For P2’s renewal clause, the TLA+ module stores no renewal base. It tests at each response that the reported ballot is the proposer’s active ballot, which within the module changes only through the attempt’s own activation, so the test is equivalent to the stored base of the executable models.

The model has a variable `now`, and executable guards such as `now < pendingDeadline` read it directly. Since this can look like an assumption of synchronized clocks, we state precisely what it means. The variable `now` is the common real-time axis on which safety is asserted, and each local timer is a duration that has been conservatively translated onto that axis using the rate bounds of T2. When a process in the model compares `now` with its own deadline, it represents an implementation reading its own monotonic elapsed-time counter, never another process’s clock. Similarly, the certificate variables `cert` and `certDeadline` that appear in the invariants are history variables. They record, for the benefit of the checked predicates, which acceptors granted the current lease and the exclusion deadlines those acceptors reported. The protocol itself never compares a deadline measured on one clock with a deadline measured on another.

Appendix B.1. Lease Invariants

TypeOK: Every variable remains in its declared finite domain: times are bounded, messages have one of the modeled shapes, acceptor state maps acceptors to lease records, proposer phases are among the modeled phases, and response sets are sets of acceptor identities. This is the well-formedness condition needed before the other predicates have their intended meaning.

AcceptedCoherence: An acceptor’s accepted value is either `NoLease` or a lease whose owner, ballot, and deadline lie in the modeled domains. This rules out malformed accepted state and ensures that later `Prepare` responses can be interpreted as lease reports.

ActiveImpliesUnexpired: Every active proposer has a local deadline strictly greater than the current model time. This captures the rule that a proposer stops acting when its own lease interval expires.

LeaseExclusivity: The set of active proposers has cardinality at most one. This is the main safety property. It is checked directly by TLC, checked by the Python simulator after every operation, and violated on purpose by every counterexample variant.

ActivationHasQuorum: An active proposer has a quorum certificate. A proposer cannot become active merely because it sent requests or received a non-quorum set of responses.

TimerContainment: For every acceptor in an active proposer’s certificate, the proposer’s deadline comes no later than the acceptor deadline recorded in the certificate. The certificate variables are history variables ([Section 3](#)), so the invariant is a statement checked by the model, not a comparison performed by the protocol.

QuarantinePreventsParticipation: An acceptor in quarantine has no promised ballot and no accepted lease. Together with the transition rules, this means that a restarted acceptor cannot vote while forgotten lease state could still matter.

ActiveBallotWellFormed: An active proposer’s active ballot is one of the modeled ballots. This is a bookkeeping invariant for the certificate machinery. The substantive renewal property, that a failed renewal does not extend authority, is a transition rule exercised by the renew/release configuration and by the scenario tests.

Appendix B.2. PaxosLease+Paxos Invariants

ChosenValueWellFormed: The abstract composition model represents each slot by a single chosen value or by `NoValue`, and the transition relation assigns a chosen value only when the slot is empty. Agreement is therefore enforced by construction in this abstraction, the named invariant checks domain membership, and the substantive checks are the admission invariants below. We state this explicitly to avoid overclaiming, since the composition model checks the lease/Paxos boundary, not Paxos itself.

ReadyImpliesActive: A ready leader holds a live lease. This is the admission side of the composition, since a process without lease authority cannot be ready to accept client log work.

ReadyLeaderUniqueness: At most one proposer is in the ready state. This is stronger than Paxos requires for agreement. In the composition model it follows from the single-holder premise of `AcquireLease` together with `ReadyImpliesActive`, so it records the property the lease layer is assumed to supply.

RecoveryPrecedesAdmission: A proposer cannot apply or admit log work before it has completed recovery. This is the modeled boundary between the lease protocol and Paxos recovery.

ReadyImpliesRecovered: A ready leader has completed recovery within its current lease epoch, since acquiring a lease clears the recovery mark. This fences the fast path per epoch: no leader appends on the strength of a previous epoch’s recovery.

AppliedImpliesChosen: Every slot applied by any proposer has a chosen value, so the leader-admission abstraction cannot invent applied log entries that Paxos has not chosen. **Log Prefix Consistency** is the two-proposer restatement of the same fact, implied by it and retained as a separate named check for readability.

Appendix B.3. TLAPS Obligations

TimerContainmentArithmetic: If the proposer starts its timer no later than each certificate acceptor starts its exclusion, and its duration is no longer, then the proposer’s authority interval is contained in the acceptor’s exclusion interval.

QuarantineCoversForgottenFacts: An attempt whose `Prepare` was sent no later than the crash is unusable D_P later, so a quarantine of at least D_P from the restart outlasts it. The argument of [Section 4](#) supplies the premise, that everything an acceptor can forget belongs to such an attempt; quarantine at least D_P therefore outlasts every attempt whose `Prepare` preceded the crash, which with A2’s rule is every attempt a forgotten fact can support. The obligation has no analogue under the step-3 rule alone; with a retry bound the analogue is the next obligation.

ImplQuarantineCoversForgottenFacts: The analogue for the implemented protocol under timely timeout dispatch: a forgotten promise anchors at a `Prepare` and is unusable R later, a forgotten accepted lease anchors at a prepare-quorum receipt and is unusable D_P later, and both anchors precede the crash, so quarantine at least $\max(D_P, R)$ covers both cases.

CapMonotone: The deadline cap that keeps the finite model bounded is monotone, so capping preserves containment.

QuarantineBoundTight: Any quarantine strictly below the proposer duration leaves an elapsed interval inside a forgotten attempt’s lifetime but outside quarantine. The two-owner traces of [Section 4](#) lie in that gap.

DriftContainmentArithmetic: If the proposer’s clock runs at rate at least r_{min} , the acceptor’s at most r_{max} , and the durations satisfy $D_P \cdot r_{max} \leq D_A \cdot r_{min}$, then real-time containment holds. This is the inequality of T2, and the same arithmetic with the quarantine in place of D_A gives the real-time quarantine bound.

Appendix C. Executions with Overwriting Acceptors

This appendix gives the executions that overwriting acceptors admit: the two of [Section 6](#) in detail, those that follow when the timer rule T1 or the renewal qualifier of P2 is weakened, and a walkthrough of the one that follows from the pseudocode of [1]. The two executions of [Section 6](#) come from directed searches whose state constraints only prune behaviors, so each is a behavior of the full specification.

Renewal across an acceptor restart. With three acceptors, a renewal and a single acceptor restart give two owners even with P2’s renewal qualifier in place. Acceptor a_2 promises ballot 2 to p_2 , then crashes, forgetting the promise, and restarts into the full quarantine. Proposer p_1 , under the lower ballot 1, collects promises from a_1 and a_3 , and a_1 then promises ballot 2 as well, which completes p_2 ’s prepare quorum with a_2 ’s forgotten promise. Acceptor a_3 accepts p_1 ’s lease, and p_2 ’s `Accept` requests reach a_1 and a_3 ; at a_3 the higher ballot overwrites p_1 ’s accepted lease. Proposer p_2 becomes active and renews under ballot 3, and a_1 and a_3 both report the exact lease being renewed, so the qualifier counts them as open. When a_2 leaves quarantine it holds no promise, and it accepts p_1 ’s delayed `Accept` request. Together with a_3 ’s earlier acceptance this completes p_1 ’s majority, and p_2 ’s renewal completes as well. The quarantine outlasted p_2 ’s first attempt, but that attempt had already completed, on a majority that included an overwritten accepted lease. TLC finds the execution in a 39-state trace (`PaxosLeaseRenewCrashSearch.cfg`), and the same search with A2’s rule exhausts its space with no violation. The reference model replays it (`renewal-over-overwritten-grant`).

Early release across an acceptor restart. Proposer p_2 acquires through a_1 and a_2 under ballot 2, above p_1 ’s ballot 1, and releases at once, with its `Accept` request and its `Release` to a_3 both still in flight. Acceptor a_2 crashes, forgetting its promise and its accepted lease, restarts and serves the full quarantine, and p_1 then acquires under ballot 1 through a_2 and a_3 . The stale `Accept` request reaches a_3 and, since ballot order permits it, reinstalls p_2 ’s released instance over p_1 ’s live lease; the stale `Release` then matches that instance exactly and clears it, together with its exclusion deadline. A fresh attempt by p_2 under ballot 3 finds no lease at a_1 , whose accepted lease it released, and none at a_3 , and its `Accept` request overwrites p_1 ’s live accepted lease at a_2 , so p_2 acquires while p_1 is still active. The renewal qualifier is not involved, since p_2 ’s last attempt is fresh and the acceptors it asks hold no accepted lease, and no quarantine length prevents the execution, since the stale messages can be delayed arbitrarily. With A2’s rule, a_3 refuses to replace p_1 ’s live lease, the stale `Release` matches nothing, and p_1 ’s lease blocks p_2 . TLC finds the execution in a 40-state trace (`PaxosLeaseReleaseCrashSearch.cfg`), and the same search with A2’s rule exhausts its space with no violation. The reference model reproduces the execution and its repair (`release-after-stale-accept`). Neither Keyspace nor ScalienDB implements early release, so this execution cannot occur in them.

Timer placement (T1). T1 starts the attempt deadline when `Prepare` is sent and stores it as state. Acceptor state is volatile, so evidence gathered before a crash must expire at a fixed time after it was created. T1 assigns t_d at P1 and checks it at P3 and P4, so the whole attempt, its unprocessed promises included, expires with it.

The pseudocode of [1] starts the timer at prepare-quorum receipt instead. The attempt then has no timer until the quorum arrives, a prepare quorum stays usable for an unbounded time, and no finite quarantine covers an attempt that has no deadline. With overwriting acceptors, TLC finds the two-owner execution at the full quarantine bound in 27 states with two acceptors, and in 25 states with three acceptors and a single crashed acceptor, the one in the intersection of the two proposers’ quorums. The walkthrough below gives the failure in implementation terms. In it, each acceptor replaces a live lease of the other proposer, and with A2’s rule the two-acceptor configuration passes over 23,777,320 distinct states.

Keyspace and ScalienDB bound the attempt with a retry timeout instead, whose deadline exists only as dispatch order: a scheduling pause runs an expired attempt’s handlers before the timeout fires, and TLC finds the two-owner execution under the shipped constants ([Section 7](#), third finding). Durable promises and Fleas’s clock-derived ballots ([Section 8](#)) also prevent the execution; the repairs are compared after the walkthrough.

The 2012 pseudocode’s timer rule. Figure 2 of [1] places “start timer” on the proposer’s lane before the `Prepare` requests leave. The pseudocode instead starts it in step 3, `Proposer::OnPrepareResponse`, when a quorum of empty `Prepare` responses has arrived, immediately before sending `Propose` requests. The proof’s prose says that the proposer starts its timer before sending `Propose` requests, which is consistent with both placements.

Under the pseudocode rule, an attempt has no deadline between sending `Prepare` and processing its prepare quorum. The original restart quarantine of the maximal lease time M outlasts forgotten exclusions and attempts whose timers have started. It cannot outlast an attempt whose timer may start arbitrarily late. No finite quarantine repairs that rule.

`tla/counterexamples/LateTimer.tla` changes only timer placement: `SendAccept` sets the pending deadline instead of `StartAcquire`. With two proposers, two acceptors that overwrite, as in [1], and $D_P = D_A = D_Q = 2$, TLC finds a 27-state violation of `LeaseExclusivity`. The execution proceeds as follows.

1. At $t = 0$, p_1 and p_2 send `Prepare` with ballots 1 and 2. Both acceptors promise in ballot order, and both proposers receive empty promise quorums. Neither proposer has started its timer.
2. Both acceptors crash, forgetting their promises. They restart and remain in quarantine until $t = 2$.
3. At $t = 1$, both proposers process their prepare quorums. Each starts its timer with deadline $t = 3$ and sends `Accept`.
4. At $t = 2$, quarantine ends. The acceptors accept ballot 1, then overwrite it with ballot 2; the crash erased the promises that would have rejected ballot 1.
5. Both proposers receive accepted quorums and activate. At $t = 2$, both own the lease until $t = 3$.

Every acceptor served its full quarantine. The execution needs no message loss or clock error. The original invariance argument omits acceptor state loss; its restart wait assumes that every attempt using forgotten state already has a bounded lifetime.

In implementation terms, ordinary scheduling delay separates receipt of the last `Prepare` response from sending the `Propose` requests. A garbage-collection pause or a preempted virtual machine, together with acceptor crash and restart, supplies the whole schedule. The proposer resumes with responses that remain acceptable under the pseudocode despite the loss of the promises they report.

A2’s acceptor-side rule prevents this, since in step 4 each acceptor replaces a live lease of the other proposer. Rule T1 prevents it as well, by storing the deadline when `Prepare` is sent and checking it before sending accepts and activating, so that unprocessed promises expire with the attempt. With overwriting acceptors, T1 and a quarantine of D_P are still not enough in general (the renewal execution of Section 6); with A2’s rule they are (Section 4). The `late-promise-reuse` Python witness reproduces the unsafe schedule; its paired test uses the prepare-time rule, rejects the expired promise quorums, and sends no `Accept`.

Durable promises also prevent the execution, at the cost of a synchronous disk write on the prepare path. An expiry token on `Prepare` responses applies the same time bound as T1. Fleese’s clock-derived ballots use stronger clock assumptions (Section 8). A separate retry deadline can bound the attempt if stored and checked in every response handler. The audited implementations use a scheduled retry callback without those checks; delayed dispatch defeats it (Appendix D). For a separate retry bound R , quarantine must cover both the retry interval and the authority interval. The implementation projection checks $D_Q \geq \max(D_P, R)$ under timely dispatch. Storing and enforcing the retry deadline removes that dependence on callback ordering.

With three acceptors and majority quorums of two, TLC finds a 25-state two-owner trace under proposer and acceptor symmetry reduction. Only the acceptor at the quorum intersection crashes; it forgets its promise, serves the full quarantine, and participates in both accept quorums. The failure therefore also occurs when one acceptor remains available to each proposer throughout. Separately, quarantine one unit below D_P yields a 24-state trace with three acceptors. The repository records these searches, `make paper-claims` checks that each recorded trace has the cited length, and `make paper-evidence-full` reruns them.

Renewal check (P2). A report of the proposer’s own lease is open only during a renewal of that exact lease. The reported ballot must equal the renewal base captured at P1; ownership alone does not qualify a response.

A proposer abandons an attempt (P5) with `Accept` requests in flight and retries under a fresh ballot, and in between a competitor on its first attempt acquires under a lower ballot. With overwriting acceptors, the late `Accept` requests overwrite the competitor’s accepted lease at every acceptor, since ballot order permits it and overwriting revokes nothing, and the retrying proposer’s prepare then finds its own lease everywhere it looks. If ownership alone makes those responses open, it completes the acquisition while the competitor is still active. TLC finds the execution in 36 states at the full quarantine bound with retry enabled (variant `StaleOwnerOpen.tla`), and no quarantine length prevents it, because the stale `Accept` request can be delayed arbitrarily. Both audited implementations ship the unqualified rule (Section 7, seventh finding). With A2’s rule the stale `Accept` request cannot replace the competitor’s accepted lease, and the same configuration with the qualifier dropped passes over 337,917,446 distinct states.

Appendix D. Audit of Keyspace and ScalienDB

This appendix gives the eight findings of Section 7 in full, and the source-level evidence behind them. The audit reads Keyspace at commit a99f24a8 (2011) and ScalienDB at commit 60978146 (2013), the final commits of both repositories.

Appendix D.1. Findings

1. **Both systems implement the unsafe timer rule of the pseudocode, but an unrelated retry timeout accidentally prevents the late-timer executions.** The rule that shipped is the *unsafe* one of the pseudocode, not the prepare-time one of the figure: `StartProposing()` sets `expireTime = Now() + duration`, so the authority clock starts when the prepare quorum is in hand, the rule that Appendix C shows unsafe with overwriting acceptors, which both systems have. What both contain instead is a mechanism [1] never mentions, an attempt-restart timeout of $R = 2$ seconds armed when `Prepare` is broadcast, whose firing re-prepares under a fresh proposal identifier and thereby discards the abandoned attempt’s responses. This is T1’s deadline enforced by control flow rather than stored as data, which Appendix C shows insufficient. Neither codebase presents it as a safety mechanism; it serves as one by accident.
2. **That accidental prevention works only under certain conditions, which the implementations accidentally satisfy.** We built a checked projection of the implementations onto their lease acquisition state machine (`tla/spec/PaxosLeaseImpl.tla`) and checked when the accidental repair works, in acquisition configurations without renewal. Under timely timeout dispatch the restart condition for the late-timer executions is

$$D_Q \geq \max(D_P, R),$$

checked at its boundary and refuted one unit below the R -dominant boundary and one unit below the common boundary $D_P = R$; the boundary run also refutes the additive hypothesis $D_Q \geq R + D_P$. The shipped ordering of the constants passes exhaustively in these configurations; the seventh finding shows the shipped protocol unsafe for another reason.

3. **The event loop’s handler ordering can produce a two-owner execution.** Neither proposer stores when its attempt began; the R bound holds only because each event-loop iteration runs due timers before socket events. The real assumption is therefore stronger than freedom from suspension: no message handler belonging to an expired attempt ever executes past the deadline before the timeout transition runs, and the loop structure does not enforce that: the poll blocks until the next timer is due, so a socket that becomes ready just before the deadline has its handlers, and any burst behind them, dispatched before the next timer scan. A process suspension beginning after a timer scan and ending inside the poll is one instance; ordinary event-loop overrun is another.

TLC finds the resulting two-owner execution with the shipped constants in 27 states with two independently crashing acceptors, and in 25 states in the colocation-valid configuration described below, where only the quorum-intersection acceptor crashes; the `suspended-event-loop` witness replays it at `IsLeaseOwner()` level. The shipped constants are therefore not enough to call the implementations safe.

4. **Quorums are counted by message rather than by acceptor identity, which is safe only because their TCP transport does not deliver a response twice.** Both systems count quorum responses without acceptor identity, so one response delivered twice counts as two votes. We found no path that retransmits a response: pending writes are discarded on disconnect, and the inspected TCP connections supply non-duplicating delivery within a connection. The scalar counting therefore rests on an application-level at-most-once assumption that is plausible for these implementations but stated and checked nowhere, and the weakened variant `ScalarQuorumCounting.tla` shows what it protects: under a transport that may re-deliver, TLC finds a two-owner execution in 18 states.
5. **Every lease timer reads the wall clock, and neither system defends both of the directions in which a wall clock is unsafe.** Clock errors are not symmetric between the roles: backward or slow steps are unsafe on a proposer (authority extends) and merely prolong exclusion on an acceptor, while forward or fast steps are unsafe on an acceptor (exclusion shrinks) and harmless on a proposer. Keyspace uses `gettimeofday()` raw and is exposed in both unsafe directions; ScalienDB repairs backward steps well, through a correction thread, but lets forward steps pass uncorrected. Neither system calls the monotonic clocks its platforms provided, `clock_gettime(CLOCK_MONOTONIC)` on Unix and `GetTickCount64` or `QueryPerformanceCounter` on Windows.
6. **The learner is where ownership becomes visible, and its remote expiry rests on an undocumented delivery bound.** Ownership is application-visible only through the learner, and the learner restarts the countdown from the moment the message arrives rather than from the moment the lease began. In both systems the owner's own learner installs the absolute expiry the proposer computed, so the two-owner executions above carry through to two simultaneously application-visible masters; but every *remote* learner sets its expiry to arrival time plus the remaining duration minus a 500 msec hedge, so the hedge is conservative only while `LearnChosen` delivery takes less than 500 milliseconds, another undocumented timing assumption.
7. **A proposer treats any lease it owns as an open response even when it is no longer the active owner.** Both proposers treat a reported lease they own as an open response whether or not they are currently the active owner: `StartProposing` proceeds, with a full fresh duration, whenever the discovered lease owner is the node itself. Appendix C shows the rule unsafe with abandon and retry (P5), of which the retry timeout of the first finding is an instance: an `Accept` request from an abandoned attempt can overwrite a competitor's live accepted lease and then serve as its sender's own justification, and TLC finds the resulting two-owner execution in 36 states at the full quarantine bound, with no dispatch delay and no clock error. The execution does not depend on connectionless delivery: under the connection-oriented refinement, in which a crash discards the messages addressed to the crashed process, TLC finds it at the same depth, with the stale `Accept` requests sent after the acceptors restart, as a reconnecting writer does. The renewal qualifier that P2 states prevents this execution: an own lease is an open response only for a renewal attempt, and only for the exact lease captured as its base while it is still held. A2's refusal prevents it as well. The composition is checked in the implementation projection itself, not only in the base model: with three ballots, the shipped timely constants, and the connection-lifecycle transport, the projection combines the shipped retry timeout with the shipped own-lease rule, and TLC finds a 35-state two-owner execution (`PaxosLeaseImplStaleOwner.cfg`). Timer dispatch is timely throughout that execution: it needs no suspension, no dis-

patch delay, and no clock error. These traces crash both acceptors of a two-acceptor system, so they model acceptors that crash independently of the proposers; whether they arise in a two-node deployment, where each node hosts both roles, is not established.

8. **The activation guard reads the clock twice, and an unsigned subtraction can wrap so that a proposer activates a lease that has already expired.** The Phase 2 handlers read the clock once for the expiry guard and again for the activation-margin subtraction, on `uint64_t` deadlines. If the clock crosses the deadline between the two reads, through a pause between the calls or a forward step, the unsigned subtraction wraps, the margin check passes, and the proposer activates an expired lease; Keyspace additionally broadcasts the wrapped value as the lease duration. The window is narrow, but it is a time-of-check to time-of-use race on the safety-critical comparison, and the repair, one clock read per handler with a compare-before-subtract, is standard.

Appendix D.2. Timer Rule and Retry Timeout

In both systems `StartProposing()` sets `expireTime = Now() + duration` (`Keyspace PLeaseProposer.cpp`, `ScalienDB PaxosLeaseProposer.cpp`). The attempt-restart timeout is `ACQUIRELEASE_TIMEOUT`, 2 seconds in both, armed by `StartPreparing()`; when it fires, `OnAcquireLeaseTimeout()` calls `StartPreparing()` again under a fresh proposal identifier. The maximal lease time is `MAX_LEASE_TIME`, 7 seconds in Keyspace and 3 in ScalienDB, and both the lease duration and the startup quarantine are set to it.

`OnProposeResponse` contains the unsigned-arithmetic race of the eighth finding. It tests `state.expireTime < Now()` and returns if the lease has expired, then, with a separate clock read, tests `state.expireTime - Now() > 500` before activating; `expireTime` is `uint64_t` (`PLeaseState.h`). A clock that crosses the deadline between the two reads wraps the subtraction to nearly 2^{64} , which passes the margin test, and Keyspace then passes `state.expireTime - Now()`, a third read, to `LearnChosen` as the lease duration. The `unsigned-expiry-underflow` witness records the arithmetic.

Appendix D.3. TLA+ Model of the Shipped Protocol

`tla/spec/PaxosLeaseImpl.tla` has overwriting acceptors, as both implementations do, and differs from the base specification with overwriting acceptors in three audited rules: `pendingDeadline` is set in `SendAccept` rather than `StartAcquire`; an attempt deadline of R units, set at `StartAcquire`, abandons the ballot when it fires; and a reported own lease counts as open whenever the proposer owns it, with no renewal base. Unlike the shipped counter-major identifiers, whose attempt counter restarts, the projection keeps a restarted proposer's attempt counter, so its ballots never decrease across a restart, and its passing configurations do not cover ballot regression after a proposer restart. It has no release, as the shipped systems have none, so the incarnation execution of Section 4 does not arise in it. The Phase 2 delivery action has no attempt-deadline guard, matching `OnProposeResponse`. The constant `TimelyDispatch` selects the timeout semantics. Under timely dispatch, advancing time abandons every overdue attempt before anything else happens, which assumes that overdue attempts are abandoned before any further response handler executes. Under delayed dispatch, expiry only enables an abandonment action that any amount of message processing may postpone, which covers both a suspension and ordinary handler overrun.

Appendix E lists the module's timer and quarantine configurations, in (D_P, R, D_Q) , with their state counts. Since the boundary passes and one unit below it fails, the condition the shipped constants must satisfy in these configurations is $R \leq M$, not $R < M$, and both implementations satisfy it strictly. The coverage lemma behind the bound is the TLAPS obligation `ImplQuarantineCoversForgottenFacts` (Appendix B).

The two-acceptor delayed-dispatch trace crashes both acceptors, and in the deployed systems a node crash restarts the colocated proposer and bumps the durable restart counter in its proposal identifiers, so in a two-node deployment that trace would change both proposers' ballot ordering.

`PaxosLeaseImplDelayedColocated.cfg` therefore uses three acceptors and restricts `CrashableAcceptors` to the single acceptor in the intersection of the two proposers' quorums. No proposer-hosting node then crashes, no restart counter moves, and the node-identity tiebreak between equal attempt counters is legitimate, so its 25-state trace is valid for the colocated deployment without modeling restart counters; the schedule below is of the same kind: only the quorum-intersection acceptor crashes, and the higher-ballot proposer's handlers run after its retry deadline.

Appendix D.4. A Two-Owner Schedule under Delayed Dispatch

A process that merely resumes after a long pause fires the overdue retry at the next timer scan and discards the stale responses, so the schedule pauses inside one loop iteration, between `RunTimers()` and the return of `IOProcessor::Poll()` (Section 7, third finding). It crashes exactly one node, the acceptor in the intersection of the two quorums, which hosts neither active proposer.

1. Node 2's proposer broadcasts `Prepare`, and acceptors 1 and 2 promise.
2. Node 1 alone crashes after promising, forgets the promise, restarts, and begins its full quarantine.
3. Node 2 processes its prepare quorum and starts proposing: its authority interval starts now, and its `Accept` requests are delayed. Its loop then performs the last timer scan before the retry deadline and blocks in the poll. The suspension begins there and outlasts node 1's quarantine.
4. Node 0's proposer completes an entire fresh acquisition against acceptors 0 and 1, and its learner installs the absolute expiry: node 0 is the application-visible master.
5. The stale `Accept` requests are accepted at node 1, overwriting node 0's accepted lease without revoking node 0's authority, and, on resume, at node 2's own acceptor. Node 2's stale ballot exceeds node 0's fresh one because the attempt counters are equal, neither node restarted, and the tie breaks on node identity, exactly the counter-major layout.
6. Node 2's queued responses are then dispatched before the overdue timeout callback, the handler finds the lease expiry unreached with more than its 500 msec margin remaining, the proposer activates, and its own learner installs the absolute expiry. Nodes 0 and 2 are now simultaneously application-visible masters.

`python/paxoslease/event_loop.py` replays this schedule against an executable abstraction of the shipped node, with proposer, acceptor, and learner colocated, as the `suspended-event-loop` witness, asserted at `IsLeaseOwner()` level; a paired run shows the same schedule refused without the suspension. Exceeding the 500 msec margin requires only a virtual-machine pause, a swap stall, a `SIGSTOP`, or a long enough handler burst.

Appendix D.5. Quorum Counting

Keyspace's proposer counts scalars (`numReceived++`, `numAccepted++`), and ScalienDB's vote object takes a node identifier but uses it only for a membership test before incrementing a scalar (`MajorityQuorum.cpp`). Pending writes are discarded on disconnect in `TCPCConn::Close` (Keyspace) and `TCPCConnection::Close` (ScalienDB). A proof of at-most-once delivery across connection replacement would also have to exclude simultaneous old and new connections to one peer, regenerated responses on reconnect, and handler re-entry, which we have not done, and Keyspace's tree contains an unused UDP transport. The implementation contract states the rule disjunctively: count by identity, or document and preserve at-most-once delivery per response.

Appendix D.6. Learners

Application code reads `IsLeaseOwner()` in `PLeaseLearner.cpp` and `PaxosLeaseLearner.cpp`, which the `LearnChosen` broadcast populates at every node, the sender included. The owner's learner installs `msg.localExpireTime`, the absolute expiry the proposer computed; every other learner installs `Now() + msg.duration - 500`. The source comments call the subtraction a conservative estimate, which it is while delivery takes less than 500 milliseconds; a `LearnChosen` delayed by $\Delta > 500$ ms leaves every remote learner believing in the owner's authority $\Delta - 500$ ms past its real end. Remote learners do not act as owner, but master lookups and election suppression read this state.

Appendix D.7. Clock Sources

In Keyspace, `Now()` is `GetMilliTimestamp()`, which is `gettimeofday()` raw, with no clamp and no correction (`Platform.cpp`; the `unsafe-clock-source` witness). In ScalienDB, every lease-layer call reaches the `Now()` of `Time.cpp`: `gettimeofday()` plus a global correction offset, with a per-thread monotonic clamp. A clock thread started unconditionally at boot (`Main.cpp`) samples the clock every 5 milliseconds and, on a backward step, permanently raises the offset so that corrected time resumes at its previous value plus one resolution step, so elapsed intervals measured across the step remain approximately correct. Forward steps pass through uncorrected in both systems. ScalienDB's Windows path in `Time.cpp` builds an elapsed-time scheme on `timeGetTime()`, milliseconds since boot, and never runs it: the enclosing `gettimeofday()` opens with `return gettimeofday_win(tv, NULL)`; and everything below that return is unreachable.

Appendix D.8. Connection-Oriented Transport

Under `CrashDropsIncoming`, messages addressed to a process are discarded when it crashes and again when it restarts, so every delivered message was sent after its addressee's most recent restart; messages a crashed process already sent survive. The verdicts do not change. The late-timer execution is unchanged at 27 states, because its stale messages are promises, sent by the acceptors before they crash. TLC finds the stale-owner execution at 36 states as well: the acceptors crash and restart first, and the abandoned attempt's `Accept` requests are sent afterward, on fresh connections, as a reconnecting writer does. In the implementation projection, the colocation-valid configuration under this transport (`PaxosLeaseImplDelayedColocatedTcp.cfg`) finds a 25-state two-owner execution, and the stale-owner projection of the seventh finding runs under the same transport. Connection teardown masks none of these executions; the transport property safety rests on is the at-most-once delivery of the fourth finding.

In neither the renewal nor the early-release execution of Section 6 is a message that was sent before an acceptor's restart delivered to that acceptor after it, so both are also executions under this transport.

Appendix E. Repository Inventory

This appendix inventories the evidence behind Section 5: every TLC configuration of the specification with its state count and verdict, the mapping from each rule to the model or test that checks it, the implementation-projection configurations, and the Python models.

Table 2 lists the configurations of `tla/spec/PaxosLease.tla`. The two boundary rows are the separated-duration pairs of Section 4, and the rows that combine renewal, retry, release, redelivery and crash are the interaction configurations of Section 5.

Inside such a configuration TLC enumerates every reachable state and finds no violation of `LeaseExclusivity`; about larger ones it says nothing, and Section 9 says which of these bounds the result actually rests on. Table 2 lists them all. They cover the separated-duration boundary pairs of Section 4, explicit retry at the quarantine boundary, and the redelivering transport. Further configurations test feature interactions, combining renewal, retry, release, redelivery, and crash, and two of them run release with two proposers. Features that pass in isolation need not pass together, as the stale-owner execution shows, so these configurations combine them. A further family of configurations projects the rules the

Configuration	(D_P, D_A, D_Q)	Purpose	Distinct states	Result
<i>The specification (PaxosLeaseChecked.tla): A2's refusal, owners compared as incarnations</i>				
Base	(2, 2, 2)	acquisition races, two proposers, three acceptors	491,037	pass
Renew/release	(2, 2, 2)	renewal and exact-instance release, one proposer	7,717	pass
Crash/restart	(2, 2, 2)	two proposers race across crash and restart	37,425,056	pass
Drift	(1, 2, 2)	drift-adjusted (shorter) proposer duration, one proposer	40,784	pass
Quarantine = D_P	(1, 2, 1)	boundary: quarantine below acceptor exclusion	76,749,192	pass
Quarantine = D_P	(2, 3, 2)	boundary at a second duration pair	36,772,096	pass
Redeliver	(2, 2, 2)	redelivering transport, identity counting, no crash	1,857,563	pass
Release race	(2, 2, 2)	release, two proposers, three acceptors	23,743,497	pass
Release+Crash [†]	(2, 2, 2)	release, two proposers, crash and restart	PENDING	PENDING
Renew+Crash, 3 acceptors [†]	(2, 2, 2)	renewal across crash and restart, symmetry reduced	PENDING	PENDING
Release+Crash, 3 acceptors [†]	(1, 1, 1)	release across crash and restart, symmetry reduced	PENDING	PENDING
Redeliver+Crash [†]	(2, 2, 2)	redelivering transport across acceptor crash and restart	PENDING	PENDING
Retry [†]	(2, 2, 2)	explicit abandon and retry, three ballots, crash	2,179,760,458	pass
Renew+Retry [†]	(2, 2, 2)	renewal and abandon together, crash	2,411,951,538	pass
Retry+Redeliver [†]	(2, 2, 2)	abandoned attempts under redelivery	53,723,103	pass
Renew+Release+Retry [†]	(2, 2, 2)	stale Phase 2 traffic around renewal and release	204,050	pass
Incarnation (directed) [†]	(2, 2, 2)	the search of the NodeOwner execution	220,119	pass
Renew+Crash (directed) [†]	(2, 2, 2)	the search of the renewal execution	6,330,050	pass
Release+Crash (directed) [†]	(1, 1, 1)	the search of the early-release execution	224,164,193	pass
Lease+Paxos	n/a	abstract composition barrier	2,095	pass
<i>The specification one unit below the quarantine bound (PaxosLease.tla)</i>				
Unsafe quarantine	(1, 2, 0)	quarantine one unit below D_P	not exhaustive	violated (25-state trace)
Unsafe quarantine	(2, 3, 1)	quarantine one unit below D_P, D_A larger	not exhaustive	violated (26-state trace)
<i>Overwriting acceptors, the rule of [1] (RefuseLiveOverwrite = FALSE)</i>				
Renew+Crash (directed) [†]	(2, 2, 2)	renewal across one restart, three acceptors	not exhaustive	violated (39-state trace)
Release+Crash (directed) [†]	(1, 1, 1)	early release across one restart	not exhaustive	violated (40-state trace)

Table 2: TLC checks of the specification. Passing counts are exhaustive for the finite configurations, subject to the per-configuration bounds on in-flight messages, on proposer restarts, and on the time horizon, which is at most two duration units in every configuration of the specification except the renewal-and-release ones, which run to three; the directed rows exhaust only the space their state constraint leaves, which only prunes behaviors. Violated runs stop at the counterexample; their trace lengths are minimum-depth under breadth-first search and stable across runs, while the number of states a parallel search happens to explore before finding the violation is not, so it is not cited. The two boundary rows establish $D_Q \geq D_P$ as the tight bound in the checked model together with the two rows one unit below it. Rows marked [†] are recorded runs, checked by `make paper-claims` and re-run by `make paper-evidence-full`; the others run in `make paper-evidence`.

Rule	Base module	Variant/projection	Python witness	Source audit
Proposer lease timeout (P1, T1)	StartAcquire stores t_d	LateTimer (overwriting)	simulator, timer witnesses	unsafe rule shipped
Retry time bound (P5)	AbandonAttempt, Retry cfg	impl projection (R)	n/a	undocumented $R \leq M$
Exact-instance release (A3)	DeliverRelease	OwnerOnlyRelease	release witness	no release path shipped
Restart quarantine (A4, T3)	RestartAcceptor	unsafe-quarantine cfgs	quarantine witness	full- M quarantine
Identity counting (P2)	response sets, Redeliver	ScalarQuorumCounting	duplicate witness	scalar counters, TCP accident
Own-lease-open qualifier (P2)	DeliverPromise	StaleOwnerOpen (overwriting)	stale-owner-open witness	unqualified rule shipped
Acceptor-side refusal (A2)	DeliverAcceptReq, SameOwner	renewal and release searches, NodeOwner	renewal, release and incarnation witnesses	overwriting rule shipped
Learner deadlines (L1)	not modeled	n/a	event-loop witness	remote re-anchoring
Clock discipline	not modeled	n/a	underflow witness	double clock read
Leader lifecycle (M1–M5)	composition model	n/a	leased_paxos.py, tests	recovery conflation

Table 3: Each rule of Section 3 and Appendix A mapped to the models and tests that check it. “Not modeled” marks the implementation-layer rules that the base transition system deliberately abstracts away; their evidence is executable witnesses and the audit of Appendix D.

two audited implementations actually ship, in particular a retry timeout in place of a stored attempt deadline, and reproduces both machine-found failures under the constants those systems ship (Section 7).

The five variants of Table 1 each weaken one rule. Three of them weaken a rule that the protocol needs with A2’s rule in place: the incarnation clause of A2 itself, in the 43-state execution of Section 4, and the two below. If release matches on owner alone, then a Release message delayed past a re-acquisition by the same owner erases the newer lease. The owner releases instance $(p_1, 1)$ and re-acquires as $(p_1, 2)$, the stale Release then clears the ballot-2 lease at the acceptors, and a second proposer acquires while p_1 is still active, in a 36-state trace. With quorums counted by message under a transport that may re-deliver, the fourth finding of Section 7, a single acceptor’s doubled response is a quorum, and two proposers become active in 18 states with no crash, no restart, and no clock movement at all. The other two, LateTimer and StaleOwnerOpen, weaken T1 and P2 with overwriting acceptors, the rule of [1] and of both audited implementations: the timer placement of

Appendix C, which walks its 27-state trace through in full, and the stale-owner execution, whose 36-state search is the multi-hour recorded run `make counterexamples-staleowner`.

The safety argument of Section 4 rests at several points on inequalities between durations, and those inequalities are proved mechanically, by TLAPS, in `tla/proof/PaxosLeaseProof.tla`: the inequalities behind timer containment, the quarantine bound and its tightness, the monotonicity of the deadline cap used by the finite model, and the clock-rate condition of T2. They are arithmetic facts; the protocol-level premises they are applied to are the argument of Section 4, checked by TLC.

The accompanying repository also contains `python/demo.py`, a dependency-free, single-file implementation of the algorithm of Section 3, written to be read by practitioners. It runs three nodes in one event loop on real monotonic-clock timers, and each node hosts a proposer, an acceptor, and a learner. Every method names the step it implements, P1 through P5, A1, A2 and A4, L1, and the timing rules, so the file reads against the paper. Where this paper shows that a plausible alternative to

a rule is unsafe, the comment at that rule says so and points at the section. The program demonstrates normal operation: a node acquires the lease, renews it at half-life, and dies; a survivor takes over once lease and quarantine have run out; and a referee object receives every learner-level ownership claim and asserts throughout that no two nodes are ever owner at once.

Every configuration bounds the number of in-flight messages with the constant `MaxNetwork`, so a passing count is exhaustive for the constrained state graph (Section 9). A violation found under the bound is a legal trace of the unbounded model, so the bound does not weaken the violated rows. The Retry configuration at bound 5 passes over `RETRYMN5` distinct states, and the violating executions of this paper need at most six concurrent messages, except the two release executions of Section 6, early release and the incarnation execution, which reach seven.

Not every rule of Section 3 and Appendix A lives in the base module, and Table 3 states which model or test checks which rule. The proposer, acceptor and timing rules are in the base module, checked in every configuration. The learner rule L1 and the clock discipline concern how an implementation surfaces activation and expiry, which the base module abstracts away, so their evidence is the executable witnesses and the source audit of Appendix D. The leader lifecycle M1 through M5 is checked by the TLA+ composition model, which checks the admission barrier between the two layers, and by the Python model, which runs the recovery, repair and append rounds against ballot-checking durable acceptors.

The implementation projection `tla/spec/PaxosLeaseImpl.tla` of Section 7 has seven timer and quarantine configurations, written (D_P, R, D_Q) , beside three transport and stale-owner projections. Three pass under timely timeout dispatch: $(D_P, R, D_Q) = (2, 2, 2)$ over 37,160,904 distinct states, $(1, 2, 2)$ over 37,476,080, and the shipped ordering $(2, 1, 2)$ over 18,160,464. Four violate `LeaseExclusivity`: quarantine one unit below the boundary, at $(2, 2, 1)$ and $(1, 2, 1)$, each in 26 states; the shipped constants under delayed dispatch with two acceptors, in 27 states; and the colocation-valid form with three acceptors, in which only the quorum-intersection acceptor may crash, in 25 states. A further projection combines the shipped retry timeout with the unqualified own-lease rule under timely dispatch and the connection-lifecycle transport, and finds a 35-state two-owner execution (Section 7, seventh finding).

The Python layer holds two deterministic models with virtual time, a lease-only model and a composed lease-plus-Paxos model. They encode the rules a second time, independently of the TLA+, and assert the safety invariants after every step of every schedule they run. TLC searches all schedules within a configuration; the Python models replay specific ones. Eighteen structured witnesses keep every known failure executable, and each must fail with its specific expected message, so an unrelated failure does not satisfy its test.

The five mechanically generated variants of Table 1 each weaken one rule, and TLC finds an exclusivity violation in each; those counterexamples are the stronger class, since there the model checker finds the violating schedule itself. TLAPS proves the six arithmetic obligations listed in Appendix B. The multi-hour searches, the rows of Table 2 marked †, the stale-owner runs, the three-acceptor counterexamples and the colocation-valid search, are recorded runs; `make paper-claims` checks their recorded state counts and trace lengths against the numbers this paper cites, and `make paper-evidence-full` re-runs them.

Appendix F. How This Paper Was Written

Large language models produced much of the TLA+ specifications and TLC configurations, the TLAPS proofs, the Python programs, the

source audit, and some of the article prose, under the author’s manual, iterative direction, with line-by-line reviews and rewrites; the overall process spanned months. Make targets automated the checking: they run the TLA+ parser, the TLC model checker, the TLAPS proof manager, and the Python tests, and record the results that the paper cites. A full verification run of the repository, including parsing, every TLC configuration and counterexample search, the TLAPS proofs, and the Python tests, takes more than 40 hours on a 4-core, 8-thread Xeon server with 64 GB of memory, most of it in the multi-billion-state searches.

Machine assistance and automation reduced the recurring work of maintaining the specification, configurations, counterexamples, Python models, and verification pipeline. Revisions were cheap enough that rerunning the checks remained practical after each change, making re-checking a natural part of the workflow. The source audit also benefited from assistance in tracing two old implementations against the model’s rules.

Every generated formal artifact is independently checkable. TLA+ parsing checks syntax, TLC checks invariants over each configured state graph, and TLAPS checks the arithmetic proof obligations. Weakened variants must produce their expected counterexamples, and regression targets check that each witness fails for the advertised reason.

References

- [1] M. Trencsényi, A. Gazsó, and H. Reinhardt. “PaxosLease: Diskless Paxos for Leases.” arXiv:1209.4187, 2012. <https://arxiv.org/abs/1209.4187>
- [2] L. Lamport. “Paxos Made Simple.” *ACM SIGACT News*, 32(4), 2001.
- [3] C. Gray and D. Cheriton. “Leases: An Efficient Fault-Tolerant Mechanism for Distributed File Cache Consistency.” *SOSP*, 1989.
- [4] D. Ongaro and J. Ousterhout. “In Search of an Understandable Consensus Algorithm.” *USENIX ATC*, 2014.
- [5] S. Chand, Y. Liu, and S. Stoller. “Formal Verification of Multi-Paxos for Distributed Consensus.” *FM*, 2016.
- [6] M. Burrows. “The Chubby Lock Service for Loosely-Coupled Distributed Systems.” *OSDI*, 2006.
- [7] T. Chandra, R. Griesemer, and J. Redstone. “Paxos Made Live: An Engineering Perspective.” *PODC*, 2007.
- [8] F. Hupfeld, B. Kolbeck, J. Stender, M. Höggqvist, T. Cortes, J. Martí, and J. Malo. “FaTLease: Scalable Fault-Tolerant Lease Negotiation with Paxos.” *HPDC*, 2008.
- [9] L. Lamport. “The Part-Time Parliament.” *ACM Transactions on Computer Systems*, 16(2), 1998.
- [10] B. Kolbeck, M. Höggqvist, J. Stender, and F. Hupfeld. “Fleaze: Lease Coordination without a Lock Server.” *IPDPS*, 2011.
- [11] U. Sharma, R. Jung, J. Tassarotti, M. F. Kaashoek, and N. Zeldovich. “Grove: a Separation-Logic Library for Verifying Distributed Systems.” *SOSP*, 2023.
- [12] I. Moraru, D. G. Andersen, and M. Kaminsky. “Paxos Quorum Leases: Fast Reads Without Sacrificing Writes.” *SoCC*, 2014.
- [13] H. Cirstea, M. A. Kuppe, B. Loillier, and S. Merz. “Validating Traces of Distributed Programs Against TLA+ Specifications.” arXiv:2404.16075, 2024.
- [14] M. Trencsényi and A. Gazsó. “Keyspace: A Consistently Replicated, Highly-Available Key-Value Store.” arXiv:1209.3913, 2012. <https://arxiv.org/abs/1209.3913>
- [15] M. Trencsényi and A. Gazsó. “ScalienDB: Designing and Implementing a Distributed Database using Paxos.” arXiv:1302.3860, 2013. <https://arxiv.org/abs/1302.3860>